

How To Build Codex Agents

Last updated: April 21, 2026

1) What you are building

A Codex agent is a task runner that can:

- read your repo context,
- follow local rules ('AGENTS.md', policies, runbooks),
- use tools (shell, MCP connectors, web when needed),
- and implement scoped engineering work safely.

There are two practical build tracks:

1. In-repo Codex agent workflow (best for engineering execution)
2. App-level API agent workflow (best for product features in your app)

2) Fast path: in-repo Codex agent workflow

Step A: Define agent behavior in repo policy files

Create or maintain:

- 'AGENTS.md' for coding rules, validation commands, and guardrails
- mission docs (for example: 'program.md', 'CHANGE_POLICY.md')
- validator scripts your agent can run every cycle

Goal: make expectations explicit so the agent executes consistently.

Step B: Add reusable capabilities via skills

Use skills for specialized workflows (example: Supabase, docs lookup, image generation).

Keep each skill focused:

- trigger conditions
- exact run steps
- fallback behavior
- constraints

Goal: reduce prompt repetition and improve reliability.

Step C: Add MCP connectors for external systems

Use MCP when the agent needs structured access to platforms (docs, GitHub, Canva, etc.).

Typical setup command pattern:

- 'codex mcp add <name> --url <server-url>'

Goal: avoid brittle scraping and give the agent typed tool calls.

Step D: Use the right runtime mode

Interactive development:

- 'codex'

Automated one-shot runs:

- 'codex exec "your task"'

Higher-autonomy automation:

- 'codex exec --full-auto "your task"'

Goal: choose interactivity level based on risk and CI context.

Step E: Add a loop for safe execution

Recommended loop:

1. Read constraints ('AGENTS.md', program docs)
2. Inspect relevant files only
3. Make minimal scoped edits
4. Run validators ('build', project scripts, policy checks)
5. Summarize exact outcomes and next steps

Goal: maintain velocity without widening blast radius.

3) App-level API agent workflow (product integration)

Step A: Use Responses API as the agent core

Use Responses API for stateful, tool-using workflows.

Step B: Select coding-capable model

For coding-focused flows, use a coding-capable model (for example 'gpt-5-codex') according to current model docs.

Step C: Add tools

Implement tool calling for:

- internal functions (CRUD, business actions),
- retrieval/search,
- external MCP resources where needed.

Step D: Add orchestration and memory

Track:

- conversation/session state,
- task state,
- tool outputs,
- retry/fallback policies.

Step E: Add safety and observability

Minimum production controls:

- authentication and authorization checks
- rate limiting
- structured error handling
- audit logs and traces
- eval runs before rollout

4) Build checklist (copy/paste)

1. Define agent rules in 'AGENTS.md'
2. Add or refine validator scripts
3. Add skills for recurring workflows
4. Add required MCP connectors
5. Test interactive run on one bounded task
6. Test non-interactive run on same bounded task
7. Add CI/autonomous mode only after validation is stable
8. Keep diffs small and reversible

5) Common mistakes to avoid

- Missing policy files, forcing behavior to depend on prompt wording
- Overly broad tasks that mix multiple objectives in one run
- Skipping validation after edits
- Letting the agent touch secrets, billing, or schema without explicit approval
- Adding many connectors before core workflow is stable

6) Suggested repo pattern

- 'AGENTS.md' -> operational rules and constraints
- 'docs/' -> runbooks and automation playbooks
- 'validators/' -> deterministic quality gates
- 'scripts/' -> repeatable operational commands
- 'TUTORIALS/' -> onboarding docs (this file)

7) Practical starter prompts

Interactive:

- "Audit the routing logic in 'src/App.tsx', propose minimal changes, then implement and run build."

Non-interactive:

- 'codex exec "Fix TypeScript errors in src/lib, run npm run build, and summarize changed files."'

8) Official docs (reference)

- Tools guide: <https://developers.openai.com/api/docs/guides/tools>

- Responses migration: <https://developers.openai.com/api/docs/guides/migrate-to-responses>
- GPT-5-Codex model docs: <https://developers.openai.com/api/docs/models/gpt-5-codex>
- Codex non-interactive mode: <https://developers.openai.com/codex/noninteractive>
- Codex SDK: <https://developers.openai.com/codex/sdk>
- Codex MCP: <https://developers.openai.com/codex/mcp>
- Codex subagents: <https://developers.openai.com/codex/subagents>
- Agents SDK guide: <https://developers.openai.com/api/docs/guides/agents-sdk>